In [1]:
```
# 📦 Step 0: Imports
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Display settings
pd.set_option("display.max_rows", 100)
pd.set_option("display.max_columns", 100)
```

In [2]:
```
# 📂 Step 1: Load Your Files (edit path if needed)

# Update the paths to where your downloaded files are
m2_raw = pd.read_csv("m2_raw.csv", skiprows=4)
cpi_raw = pd.read_csv("cpi_raw.csv", skiprows=4)
gdp_raw = pd.read_csv("gdp_raw.csv", skiprows=4)
```

In [3]:
```
# 🔍 Step 2: Inspect Structure
print("M2:")
print(m2_raw.shape)
display(m2_raw.head(3))

print("CPI:")
print(cpi_raw.shape)
display(cpi_raw.head(3))

print("GDP:")
print(gdp_raw.shape)
display(gdp_raw.head(3))
```

```
M2:
(266, 70)
```

	Country Name	Country Code	Indicator Name	Indicator Code	1960	1961	
0	Aruba	ABW	Broad money (current LCU)	FM.LBL.BMNY.CN	NaN	NaN	
1	Africa Eastern and Southern	AFE	Broad money (current LCU)	FM.LBL.BMNY.CN	NaN	NaN	
2	Afghanistan	AFG	Broad money (current LCU)	FM.LBL.BMNY.CN	3.255000e+09	3.590000e+09	4.23(

```
CPI:
(266, 70)
```

	Country Name	Country Code	Indicator Name	Indicator Code	1960	1961	1962	1963	1964	1965
0	Aruba	ABW	Consumer price index (2010 = 100)	FP.CPI.TOTL	NaN	NaN	NaN	NaN	NaN	NaN
1	Africa Eastern and Southern	AFE	Consumer price index (2010 = 100)	FP.CPI.TOTL	NaN	NaN	NaN	NaN	NaN	NaN
2	Afghanistan	AFG	Consumer price index (2010 = 100)	FP.CPI.TOTL	NaN	NaN	NaN	NaN	NaN	NaN

GDP:
(266, 70)

	Country Name	Country Code	Indicator Name	Indicator Code	1960	1961	
0	Aruba	ABW	GDP (constant 2015 US$)	NY.GDP.MKTP.KD	NaN	NaN	
1	Africa Eastern and Southern	AFE	GDP (constant 2015 US$)	NY.GDP.MKTP.KD	1.543153e+11	1.549618e+11	1.67261
2	Afghanistan	AFG	GDP (constant 2015 US$)	NY.GDP.MKTP.KD	NaN	NaN	

In [4]:
```
# 🧼 Step 3: Tidy Function to Clean All Datasets

def clean_world_bank(df_raw, value_name):
    """Convert wide WB CSV into tidy format: country, year, value"""
    df = df_raw.drop(columns=["Indicator Name", "Indicator Code"], errors='i
    df = df.melt(id_vars=["Country Name", "Country Code"], var_name="year",

    df = df[df["year"].str.isnumeric()]
    df["year"] = df["year"].astype(int)
    df[value_name] = pd.to_numeric(df[value_name], errors="coerce")
    df = df[df[value_name].notna()]
    return df
```

```
# Apply to all 3 datasets
df_m2 = clean_world_bank(m2_raw, "m2")
df_cpi = clean_world_bank(cpi_raw, "cpi")
df_gdp = clean_world_bank(gdp_raw, "gdp")
```

In [5]:
```
# 🔍 Step 4: Inspect Cleaned Results
print("M2")
display(df_m2.head())

print("CPI")
display(df_cpi.head())

print("GDP")
display(df_gdp.head())
```

M2

	Country Name	Country Code	year	m2
2	Afghanistan	AFG	1960	3.255000e+09
9	Argentina	ARG	1960	2.780000e-02
13	Australia	AUS	1960	7.400800e+09
28	Bolivia	BOL	1960	4.345000e+02
29	Brazil	BRA	1960	2.538182e-04

CPI

	Country Name	Country Code	year	cpi
13	Australia	AUS	1960	7.960458
14	Austria	AUT	1960	17.824167
17	Belgium	BEL	1960	15.596081
19	Burkina Faso	BFA	1960	10.713301
28	Bolivia	BOL	1960	0.000022

GDP

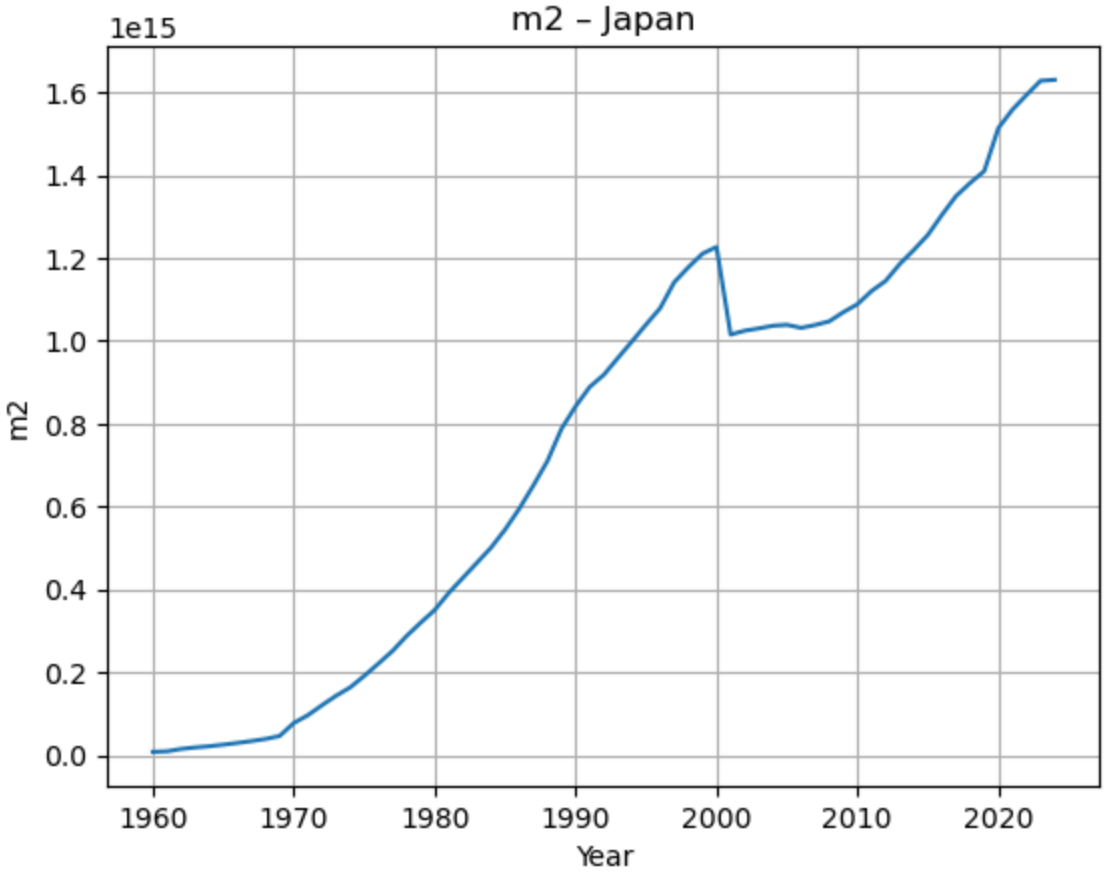
	Country Name	Country Code	year	gdp
1	Africa Eastern and Southern	AFE	1960	1.543153e+11
3	Africa Western and Central	AFW	1960	1.103632e+11
9	Argentina	ARG	1960	1.507978e+11
13	Australia	AUS	1960	2.050878e+11
14	Austria	AUT	1960	8.382867e+10

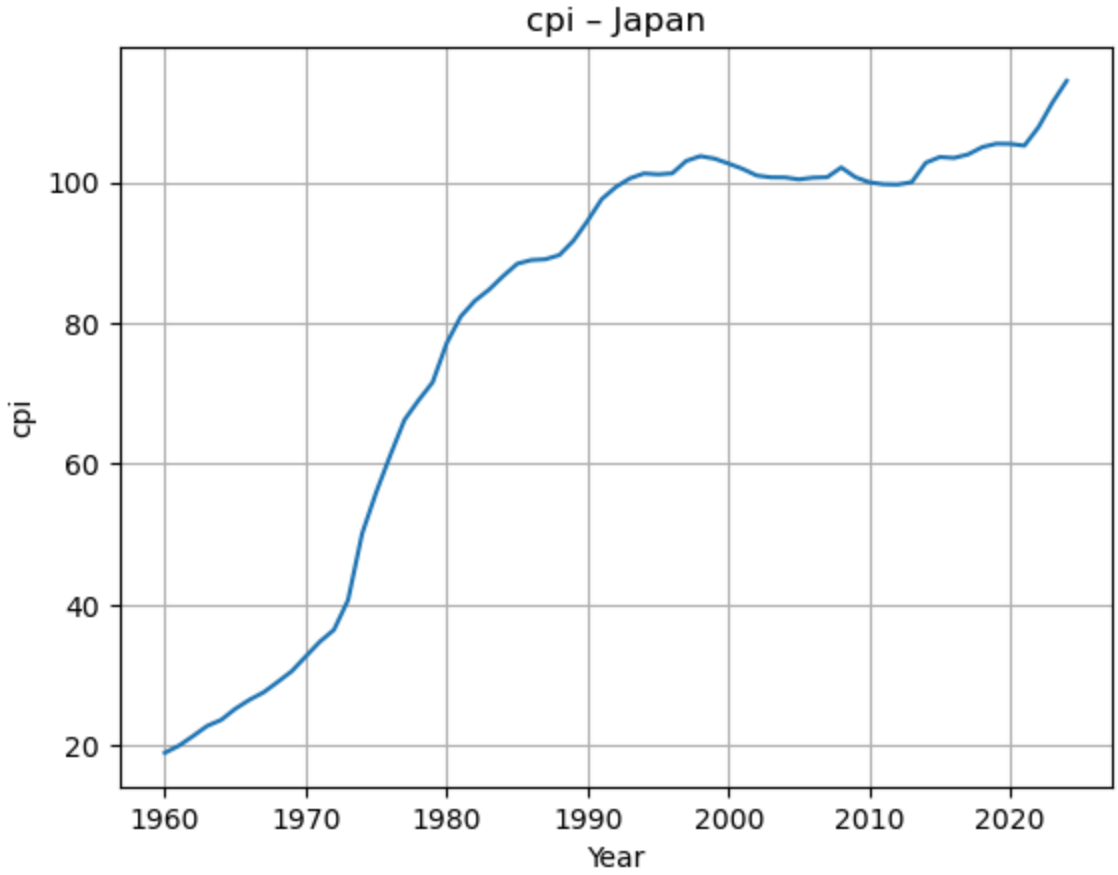
In [6]:
```
# 🔎 Step 5: Optional – Countries in Common?

m2_countries = set(df_m2["Country Name"].unique())
cpi_countries = set(df_cpi["Country Name"].unique())
```

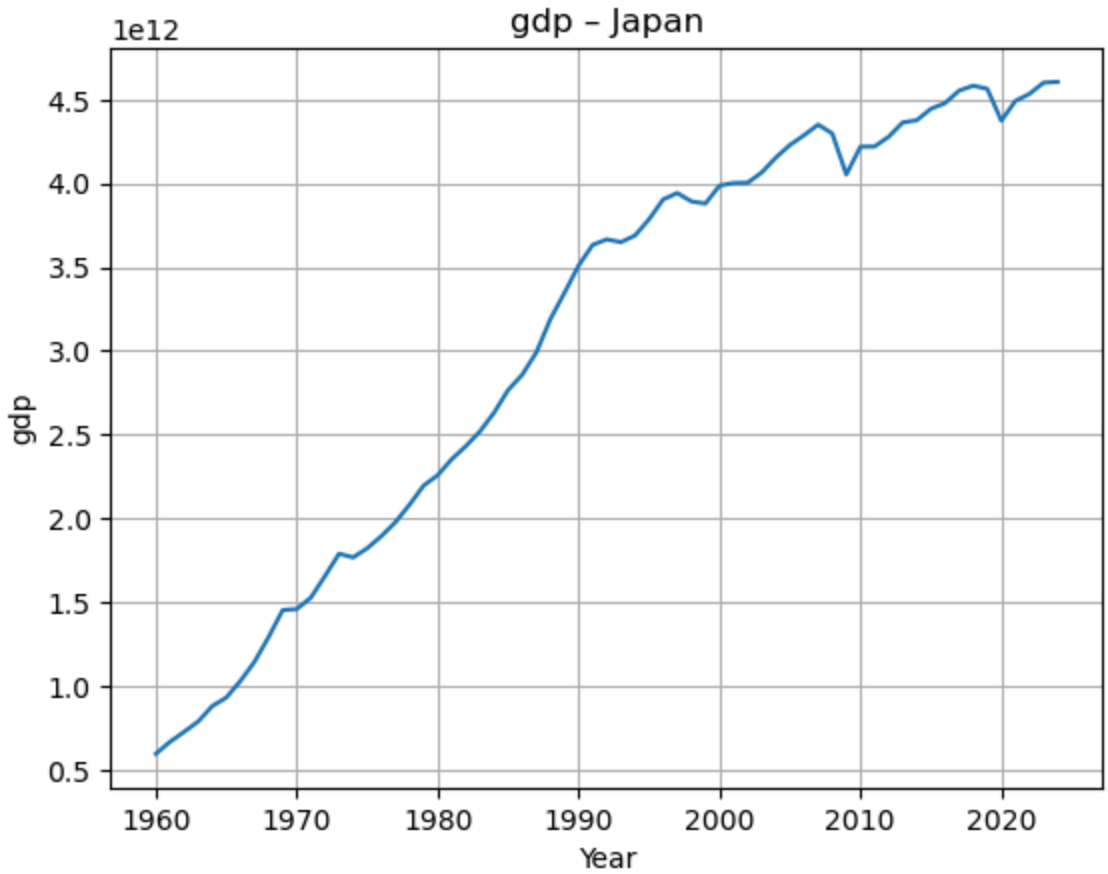
```
gdp_countries = set(df_gdp["Country Name"].unique())

common_countries = m2_countries & cpi_countries & gdp_countries
print(f"Common countries: {len(common_countries)}")
```

```
Common countries: 163
```

In [7]:
```
# 📊 Step 6: Plot Sample Country (optional)
def plot_country_trends(df, varname, country):
    temp = df[df["Country Name"] == country]
    plt.plot(temp["year"], temp[varname])
    plt.title(f"{varname} – {country}")
    plt.xlabel("Year")
    plt.ylabel(varname)
    plt.grid()
    plt.show()
x = "Japan"
plot_country_trends(df_m2, "m2", x)
plot_country_trends(df_cpi, "cpi", x)
plot_country_trends(df_gdp, "gdp", x)
```




cpi – Japan
cpi
Year
20
40
60
80
100
1960
1970
1980
1990
2000
2010
2020


gdp – Japan
1e12
gdp
Year
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
1960
1970
1980
1990
2000
2010
2020

In [36]:
```
##plot_country_trends(df_cpi, "cpi", "Brazil")
##plot_country_trends(df_cpi, "cpi", "Germany")
##plot_country_trends(df_cpi, "cpi", "South Africa")
```

In [8]:
```
# Force 1960–1990 only
YEAR_START = 1990
YEAR_END = 2020

df_m2 = df_m2[(df_m2["year"] >= YEAR_START) & (df_m2["year"] <= YEAR_END)]
df_cpi = df_cpi[(df_cpi["year"] >= YEAR_START) & (df_cpi["year"] <= YEAR_END
df_gdp = df_gdp[(df_gdp["year"] >= YEAR_START) & (df_gdp["year"] <= YEAR_END
```

In [9]:
```
def compute_log_growth(df, value_col, new_colname):
    """
    Compute log-difference growth rates per country
    """
    df = df.copy()
    df.sort_values(by=["Country Name", "year"], inplace=True)
    df[new_colname] = df.groupby("Country Name")[value_col].transform(lambda
    return df.dropna(subset=[new_colname])
```

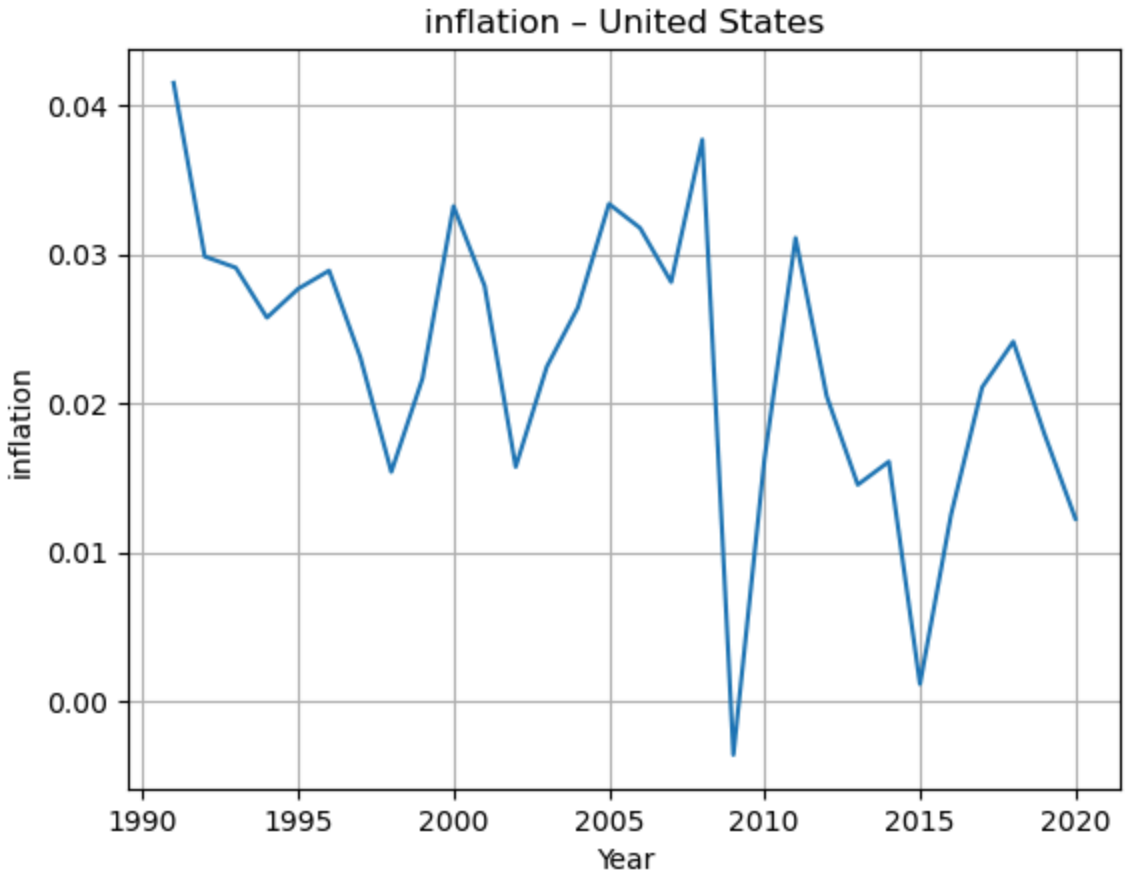
In [10]:
```
df_m2_growth  = compute_log_growth(df_m2, "m2", "m2_growth")
df_cpi_growth = compute_log_growth(df_cpi, "cpi", "inflation")
df_gdp_growth = compute_log_growth(df_gdp, "gdp", "gdp_growth")
```

In [11]:
```
df_cpi_growth[df_cpi_growth["Country Name"] == "United States"].head(10)
```

Out[11]:

	Country Name	Country Code	year	cpi	inflation
8497	United States	USA	1991	62.457341	0.041477
8763	United States	USA	1992	64.349061	0.029839
9029	United States	USA	1993	66.248425	0.029089
9295	United States	USA	1994	67.975813	0.025740
9561	United States	USA	1995	69.882820	0.027668
9827	United States	USA	1996	71.931229	0.028891
10093	United States	USA	1997	73.612758	0.023108
10359	United States	USA	1998	74.755433	0.015404
10625	United States	USA	1999	76.391102	0.021644
10891	United States	USA	2000	78.970721	0.033211

In [12]:
```
plot_country_trends(df_cpi_growth, "inflation", "United States")
```



In [13]:
```
# First merge M2 + CPI
df_merged = pd.merge(df_m2_growth[["Country Name", "year", "m2_growth"]],
                     df_cpi_growth[["Country Name", "year", "inflation"]],
                     on=["Country Name", "year"],
                     how="inner")

# Then merge GDP
df_merged = pd.merge(df_merged,
                     df_gdp_growth[["Country Name", "year", "gdp_growth"]],
                     on=["Country Name", "year"],
                     how="inner")
```

In [14]:
```
print(df_merged.shape)
df_merged.head()
```

```
(4292, 5)
```

Out[14]:

	Country Name	year	m2_growth	inflation	gdp_growth
0	Afghanistan	2006	-1.586746	0.065644	0.052188
1	Afghanistan	2007	0.353436	0.083243	0.129504
2	Afghanistan	2008	0.272953	0.234429	0.038499
3	Afghanistan	2009	0.285518	-0.070542	0.193843
4	Afghanistan	2010	0.238600	0.021551	0.134203

In [15]:
```
df_merged.to_csv("macro_growth_merged.csv", index=False)
```

In [16]:
```
import seaborn as sns
import matplotlib.pyplot as plt
import statsmodels.api as sm
```

In [17]:
```
print("M2 vs Inflation correlation:", df_merged["m2_growth"].corr(df_merged[
print("M2 vs GDP Growth correlation:", df_merged["m2_growth"].corr(df_merged
```

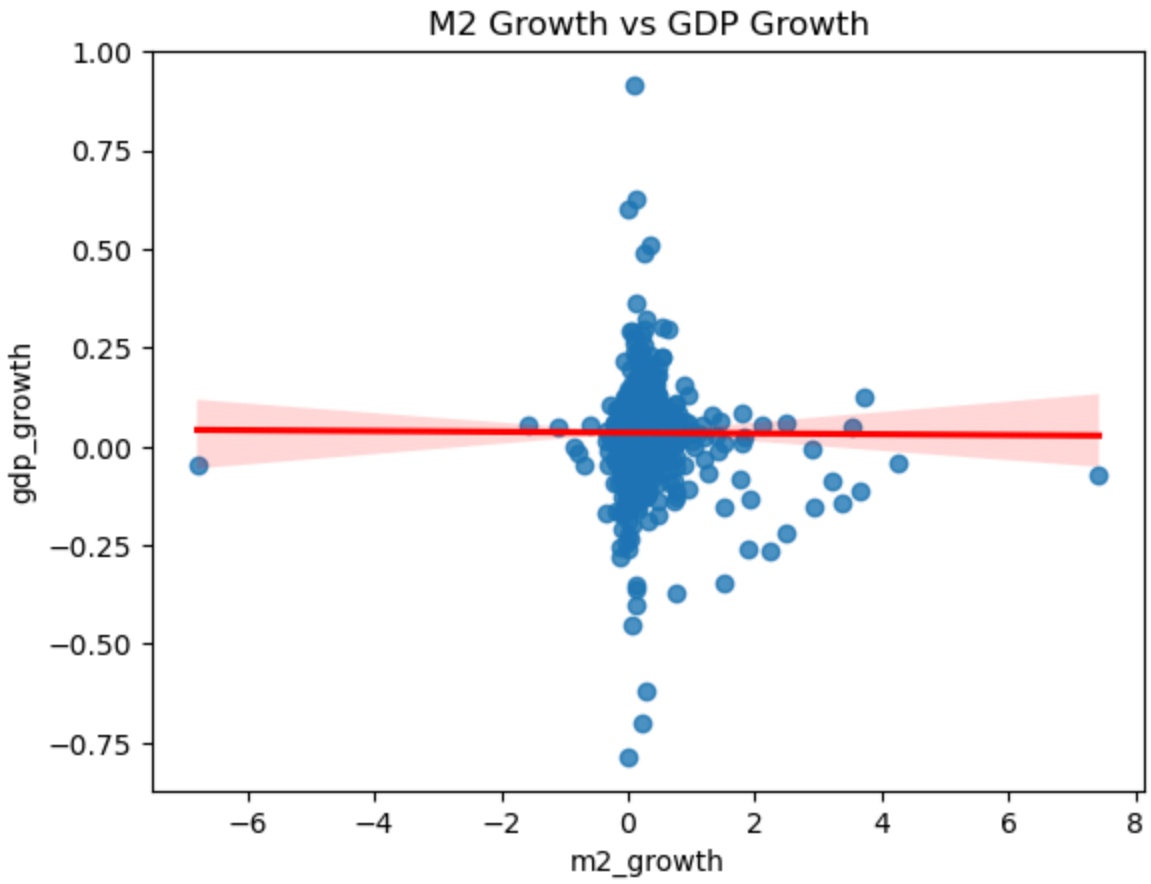
```
M2 vs Inflation correlation: 0.7243359933396607
M2 vs GDP Growth correlation: -0.004847572506673316
```

In [18]:
```
# M2 → Inflation
X1 = sm.add_constant(df_merged["m2_growth"])
y1 = df_merged["inflation"]
model1 = sm.OLS(y1, X1).fit()
print(model1.summary())

# M2 → GDP
X2 = sm.add_constant(df_merged["m2_growth"])
y2 = df_merged["gdp_growth"]
model2 = sm.OLS(y2, X2).fit()
print(model2.summary())
```

```
                            OLS Regression Results
==============================================================================
==
Dep. Variable:              inflation   R-squared:                       0.5
25
Model:                            OLS   Adj. R-squared:                  0.5
25
Method:                 Least Squares   F-statistic:                     473
5.
Date:                Wed, 07 Jan 2026   Prob (F-statistic):               0.
00
Time:                        17:43:30   Log-Likelihood:                153
4.8
No. Observations:                4292   AIC:                            -306
6.
Df Residuals:                    4290   BIC:                            -305
3.
Df Model:                           1
Covariance Type:            nonrobust
==============================================================================
==
                 coef    std err          t      P>|t|      [0.025      0.97
5]
------------------------------------------------------------------------------
--
const         -0.0073      0.003     -2.471      0.014      -0.013      -0.0
01
m2_growth      0.6451      0.009     68.813      0.000       0.627       0.6
63
==============================================================================
==
Omnibus:                     6028.423   Durbin-Watson:                   1.4
64
Prob(Omnibus):                  0.000   Jarque-Bera (JB):          5944386.9
08
Skew:                           7.658   Prob(JB):                         0.
00
Kurtosis:                     184.673   Cond. No.                         3.
71
==============================================================================
==

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is corre
ctly specified.
                            OLS Regression Results
==============================================================================
==
Dep. Variable:              gdp_growth   R-squared:                       0.0
00
Model:                            OLS   Adj. R-squared:                 -0.0
00
Method:                 Least Squares   F-statistic:                    0.10
08
Date:                Wed, 07 Jan 2026   Prob (F-statistic):              0.7
51
```

```
Time:                        17:43:30   Log-Likelihood:                 616
3.9
No. Observations:                4292   AIC:                        -1.232e+
04
Df Residuals:                    4290   BIC:                        -1.231e+
04
Df Model:                           1
Covariance Type:            nonrobust
==============================================================================
==
                 coef    std err          t      P>|t|      [0.025      0.97
5]
------------------------------------------------------------------------------
--
const          0.0349      0.001     34.959      0.000       0.033       0.0
37
m2_growth     -0.0010      0.003     -0.318      0.751      -0.007       0.0
05
==============================================================================
==
Omnibus:                     1711.017   Durbin-Watson:                   1.4
62
Prob(Omnibus):                  0.000   Jarque-Bera (JB):           360948.1
71
Skew:                          -0.731   Prob(JB):                         0.
00
Kurtosis:                      47.902   Cond. No.                         3.
71
==============================================================================
==

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is corre
ctly specified.
```

In [19]:
```
# Plot 1 – M2 vs Inflation
sns.regplot(x="m2_growth", y="inflation", data=df_merged, line_kws={"color":
plt.title("M2 Growth vs Inflation")
plt.show()

# Plot 2 – M2 vs GDP Growth
sns.regplot(x="m2_growth", y="gdp_growth", data=df_merged, line_kws={"color"
plt.title("M2 Growth vs GDP Growth")
plt.show()
```



In [20]:
```
df_merged.to_csv("final_data_for_lucas_test.csv", index=False)
```

Lucas-Style 30-Year Averages (Long-Run Cross-Country)

For each country, compute average M2 growth, average inflation, average real GDP growth over a consistent ~30-year period. Problem: Not all countries have full 1960–1990 or 1994–2024 data Fix: Dynamic 30-Year Window Per Country 1. Only include countries with at least 25+ years of valid data 2. Use each country's longest 30-year span, or close to it 3. Compute means only within that window

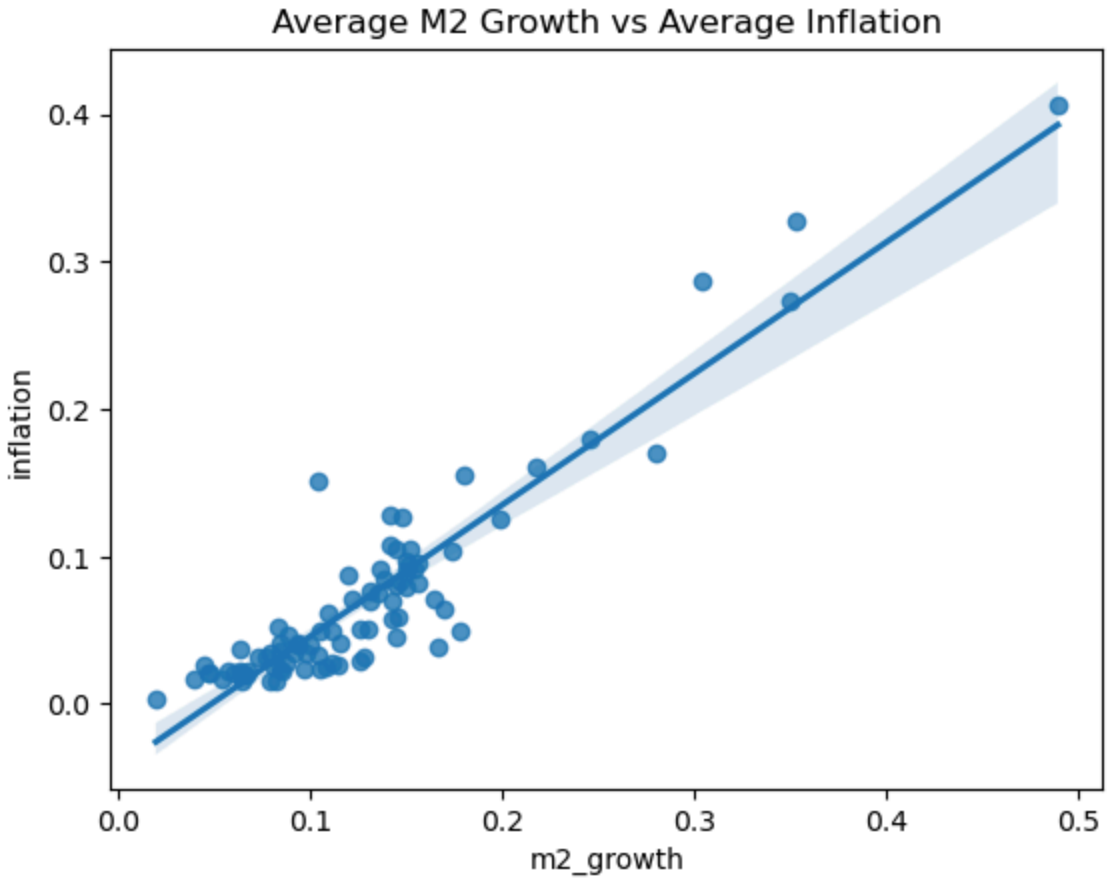
In [21]:
```
def compute_country_averages(df, varlist, min_years=25):
    """
    Computes per-country long-run averages over available data,
    filtering out countries with too few observations.
    """
    results = []

    for country, group in df.groupby("Country Name"):
        if group.shape[0] >= min_years:
            avg_vals = group[varlist].mean()
            avg_vals["Country Name"] = country
            avg_vals["n_obs"] = group.shape[0]
            results.append(avg_vals)

    df_avg = pd.DataFrame(results)
    return df_avg.reset_index(drop=True)
```

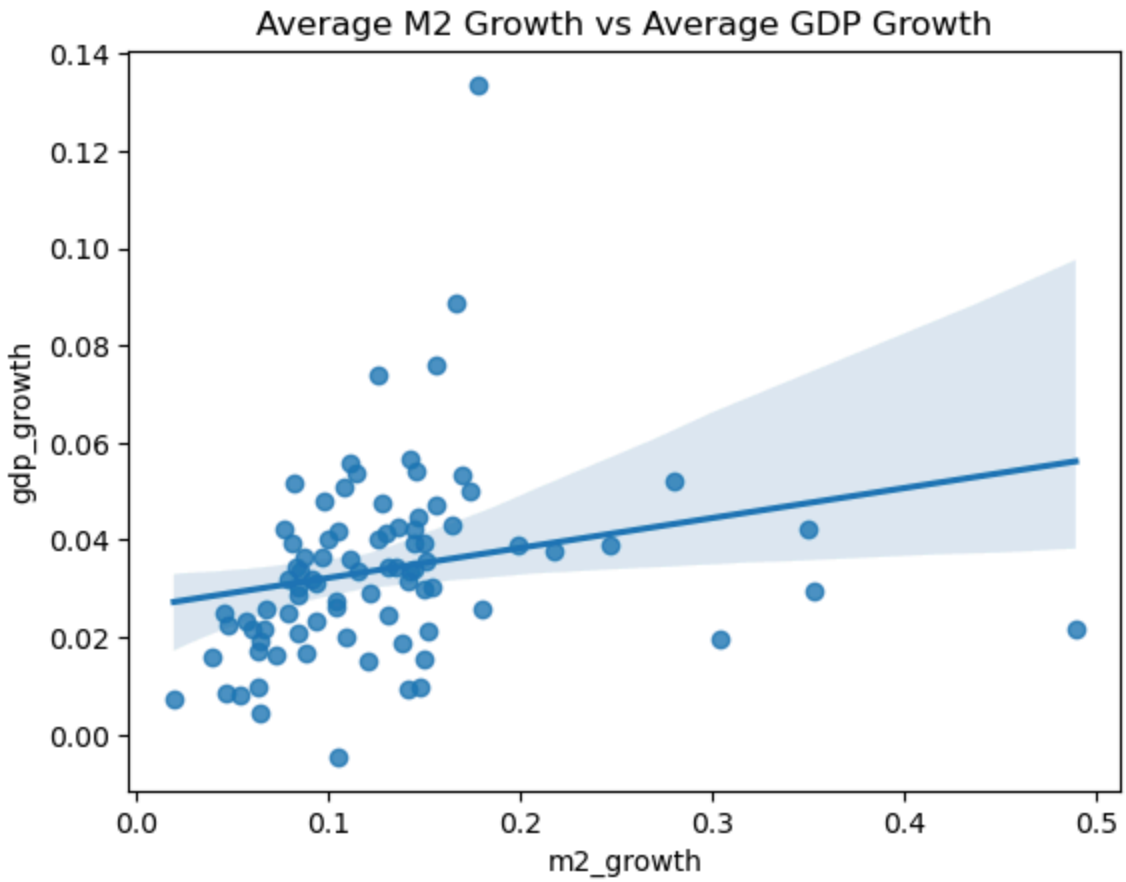
In [22]:
```
vars_to_avg = ["m2_growth", "inflation", "gdp_growth"]
df_lucas = compute_country_averages(df_merged, vars_to_avg, min_years=30)
df_lucas.head()
```

Out[22]:

	m2_growth	inflation	gdp_growth	Country Name	n_obs
0	0.131497	0.076715	0.024738	Algeria	30
1	0.084024	0.023451	0.028805	Australia	30
2	0.064016	0.020676	0.004584	Bahamas, The	30
3	0.145428	0.059101	0.053973	Bangladesh	30
4	0.078954	0.015608	0.031833	Belize	30

In [23]:
```
sns.regplot(data=df_lucas, x="m2_growth", y="inflation")
plt.title("Average M2 Growth vs Average Inflation")
plt.show()

sns.regplot(data=df_lucas, x="m2_growth", y="gdp_growth")
plt.title("Average M2 Growth vs Average GDP Growth")
plt.show()
```





In [24]:
```
df_lucas.describe()
```

Out[24]:

	m2_growth	inflation	gdp_growth	n_obs
count	86.000000	86.000000	86.000000	86.0
mean	0.128399	0.071197	0.033950	30.0
std	0.072560	0.069511	0.019303	0.0
min	0.019544	0.003645	-0.004546	30.0
25%	0.084052	0.027007	0.021726	30.0
50%	0.117596	0.049187	0.032734	30.0
75%	0.149207	0.089746	0.042043	30.0
max	0.489228	0.405775	0.133628	30.0

In [25]:

```
import statsmodels.api as sm

X = sm.add_constant(df_lucas["m2_growth"])
y1 = df_lucas["inflation"]
y2 = df_lucas["gdp_growth"]

model1 = sm.OLS(y1, X).fit()
print(model1.summary())

model2 = sm.OLS(y2, X).fit()
print(model2.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:              inflation   R-squared:                       0.8
65
Model:                            OLS   Adj. R-squared:                  0.8
63
Method:                 Least Squares   F-statistic:                     53
7.9
Date:                Wed, 07 Jan 2026   Prob (F-statistic):           2.84e-
38
Time:                        17:44:02   Log-Likelihood:                193.
86
No. Observations:                  86   AIC:                            -38
3.7
Df Residuals:                      84   BIC:                            -37
8.8
Df Model:                           1
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.97
5]
------------------------------------------------------------------------------
const         -0.0432      0.006     -7.635      0.000      -0.054      -0.0
32
m2_growth      0.8909      0.038     23.192      0.000       0.815       0.9
67
==============================================================================
Omnibus:                       12.897   Durbin-Watson:                   1.7
32
Prob(Omnibus):                  0.002   Jarque-Bera (JB):               26.0
35
Skew:                           0.480   Prob(JB):                     2.22e-
06
Kurtosis:                       5.519   Cond. No.                        1
4.1
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is corre
ctly specified.
                            OLS Regression Results
==============================================================================
Dep. Variable:             gdp_growth   R-squared:                       0.0
54
Model:                            OLS   Adj. R-squared:                  0.0
42
Method:                 Least Squares   F-statistic:                     4.7
53
Date:                Wed, 07 Jan 2026   Prob (F-statistic):             0.03
20
```

```
Time:                        17:44:02   Log-Likelihood:                220.
32
No. Observations:                86   AIC:                             -43
6.6
Df Residuals:                    84   BIC:                             -43
1.7
Df Model:                         1
Covariance Type:            nonrobust
==============================================================================
==
                 coef    std err          t      P>|t|      [0.025      0.97
5]
------------------------------------------------------------------------------
--
const          0.0260      0.004      6.263      0.000       0.018       0.0
34
m2_growth      0.0616      0.028      2.180      0.032       0.005       0.1
18
==============================================================================
==
Omnibus:                       51.229   Durbin-Watson:                   2.4
52
Prob(Omnibus):                  0.000   Jarque-Bera (JB):              238.0
66
Skew:                           1.810   Prob(JB):                     2.02e-
52
Kurtosis:                      10.303   Cond. No.                         1
4.1
==============================================================================
==

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is corre
ctly specified.
```

In [29]:
```
df_diag = (
    df_merged
    .groupby("Country Name")
    .agg(
        n_obs=("year", "count"),
        year_start=("year", "min"),
        year_end=("year", "max"),
        m2_growth=("m2_growth", "mean"),
        inflation=("inflation", "mean"),
        gdp_growth=("gdp_growth", "mean")
    )
    .reset_index()
)
print(df_diag)
df_diag.to_csv("country_diagnostics.csv", index=False)
```

```
            Country Name  n_obs  year_start  year_end  m2_growth  inflation
\
0            Afghanistan     15        2006      2020   0.024760   0.052907
1                Albania     26        1995      2020   0.116332   0.044735
2                Algeria     30        1991      2020   0.131497   0.076715
3                 Angola     25        1996      2020   0.586258   0.536188
4    Antigua and Barbuda     22        1999      2020   0.044284   0.017197
..                   ...    ...         ...       ...        ...        ...
158             Viet Nam     25        1996      2020   0.221264   0.057424
159   West Bank and Gaza     22        1999      2020   0.084056   0.026688
160          Yemen, Rep.     23        1991      2013   0.163819   0.158058
161               Zambia     29        1991      2020   0.287663   0.208806
162             Zimbabwe     11        2010      2020   0.454621   0.303177

     gdp_growth
0      0.056212
1      0.040573
2      0.024738
3      0.051862
4      0.010682
..          ...
158    0.063137
159    0.030416
160    0.039299
161    0.043038
162    0.043925

[163 rows x 7 columns]
```

In [34]:

```python
from linearmodels.panel import PanelOLS

# Make sure your panel is multi-indexed
df_panel = df_merged.set_index(["Country Name", "year"])

# Model 1: Inflation ~ M2 + Country FE + Year FE
mod1 = PanelOLS.from_formula(
    "inflation ~ m2_growth + EntityEffects + TimeEffects",
    data=df_panel
)
print(mod1.fit().summary)

# Model 2: GDP ~ M2 + Country FE + Year FE
mod2 = PanelOLS.from_formula(
    "gdp_growth ~ m2_growth + EntityEffects + TimeEffects",
    data=df_panel
)
print(mod2.fit().summary)
```

```
                          PanelOLS Estimation Summary
================================================================================
Dep. Variable:              inflation   R-squared:                        0.4312
Estimator:                   PanelOLS   R-squared (Between):              0.8655
No. Observations:                4292   R-squared (Within):               0.4501
Date:                Wed, Jan 07 2026   R-squared (Overall):              0.5729
Time:                        17:58:21   Log-likelihood                   1873.9
Cov. Estimator:            Unadjusted
                                        F-statistic:                      3107.4
Entities:                         163   P-value                           0.0000
Avg Obs:                       26.331   Distribution:                  F(1,4099)
Min Obs:                       4.0000
Max Obs:                       30.000   F-statistic (robust):             3107.4
                                        P-value                           0.0000
Time periods:                      30   Distribution:                  F(1,4099)
Avg Obs:                       143.07
Min Obs:                       113.00
Max Obs:                       156.00

                             Parameter Estimates
==============================================================================
            Parameter  Std. Err.     T-stat    P-value    Lower CI    Upper CI
------------------------------------------------------------------------------
m2_growth      0.5706     0.0102     55.744     0.0000      0.5506      0.5907
==============================================================================

F-test for Poolability: 3.6743
P-value: 0.0000
Distribution: F(191,4099)

Included effects: Entity, Time
                          PanelOLS Estimation Summary
================================================================================
Dep. Variable:             gdp_growth   R-squared:                        0.0014
Estimator:                   PanelOLS   R-squared (Between):             -0.0500
No. Observations:                4292   R-squared (Within):               0.
```

```
0004
Date:                Wed, Jan 07 2026   R-squared (Overall):             -0.
0193
Time:                        17:58:21   Log-likelihood                    67
46.6
Cov. Estimator:            Unadjusted
                                        F-statistic:                      5.
8572
Entities:                         163   P-value                           0.
0156
Avg Obs:                       26.331   Distribution:                 F(1,4
099)
Min Obs:                       4.0000
Max Obs:                       30.000   F-statistic (robust):             5.
8572
                                        P-value                           0.
0156
Time periods:                     30    Distribution:                 F(1,4
099)
Avg Obs:                       143.07
Min Obs:                       113.00
Max Obs:                       156.00

                            Parameter Estimates
==============================================================================
==
             Parameter  Std. Err.     T-stat    P-value    Lower CI    Upper
CI
------------------------------------------------------------------------------
--
m2_growth     -0.0080     0.0033    -2.4202     0.0156     -0.0144     -0.00
15
==============================================================================
==

F-test for Poolability: 6.6953
P-value: 0.0000
Distribution: F(191,4099)

Included effects: Entity, Time
```

In [37]:
```python
# Save regression-ready panel dataset
df_panel_reset = df_panel.reset_index()  # remove MultiIndex to export
df_panel_reset.to_csv("lucas_panel_dataset.csv", index=False)

# Optional: also save the notebook (from Jupyter menu manually)
# OR save as .py script if exporting to GitHub / reproducible script
```